Security

Professor G. Sandoval

Some slides derived from : Tanenbaum/Bo, and Brendan Dolan-Gavitt
Thanks !!

Security

- The Security Environment
- OS Security
- Controlling Access to Resources
- Formal Models of Secure Systems
- Basics of Cryptography
- Authentication
- Software Security

Computer Security

- Security is a big field, encompassing:
  - Application security
  - Network security
  - Authentication
  - Digital forensics
  - Cryptography
  - Privacy/Anonymity

Threat Modeling

- It usually doesn't make sense to talk about a system being "secure" or "insecure"
- Instead, we need to be more precise:
  - What are we trying to protect?
  - Who do we need to protect against? What are their capabilities?

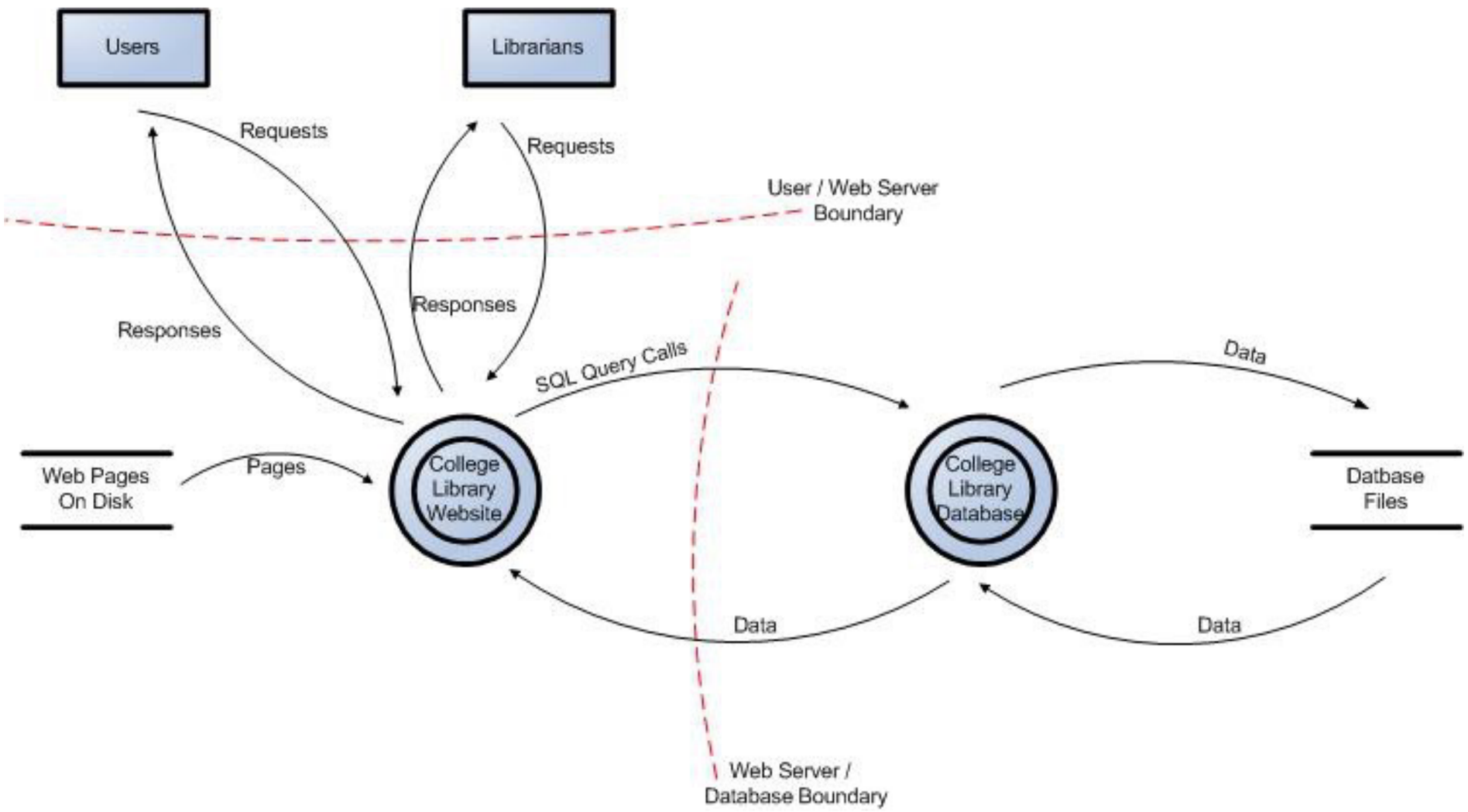
Threat Modeling

- Roughly, we can divide this into three steps:
  - System understanding
  - Threat categorization
  - Countermeasures and mitigation

System Understanding

- To properly protect a system, you need to understand it:
  - Identify assets that need protecting
  - Look at ways the system can be used and assets can be accessed
  - Figure out what rights should be granted to what assets and classes of user
  - Identify *privilege boundaries* – places where a program or user changes their privilege level

Example System Diagram



Source: OWASP

Threat Categorization

- Look at the system from an attacker's point of view
- What goals might an attacker have?
- How could they achieve these goals?
- May help to use a threat categorization such as **STRIDE**: **S**poofing, **T**ampering, **R**epudiation, **I**nformation Disclosure, **D**enial of Service, **E**levation of Privilege

Mitigation

- For each of the threats to some asset, come up with a plan for mitigating or nullifying the threat, for example:
  - Attacker might guess someone's password => enforce password complexity requirements
  - Attacker might snoop on network traffic => encrypt data that is sent on the network

Operating Systems Security

- In this course we will only worry about security as it applies to operating systems
- This is some mix of *authentication*, *access control*, and *application security*

Computer Security

- Generally, we talk about computer security in three broad categories:
  - **Confidentiality** – Exposure of data or preventing others from finding out information we don't want them to have
  - **Integrity** – Tampering with Data or preventing others from modifying our data without permission
  - **Availability** – Denial of Service or preventing others from denying us access to some service ()

Computer Security

- Generally, we talk about computer security in three broad categories:
  - **Confidentiality** – Exposure of data or preventing others from finding out information we don't want them to have
  - **Integrity** – Tampering with Data or preventing others from modifying our data without permission
  - **Availability** – Denial of Service or preventing others from denying us access to some service ()

Threat Modeling

- It usually doesn't make sense to talk about a system being "secure" or "insecure"
- Instead, we need to be more precise:
  - What are we trying to protect?
  - Who do we need to protect against? What are their capabilities?

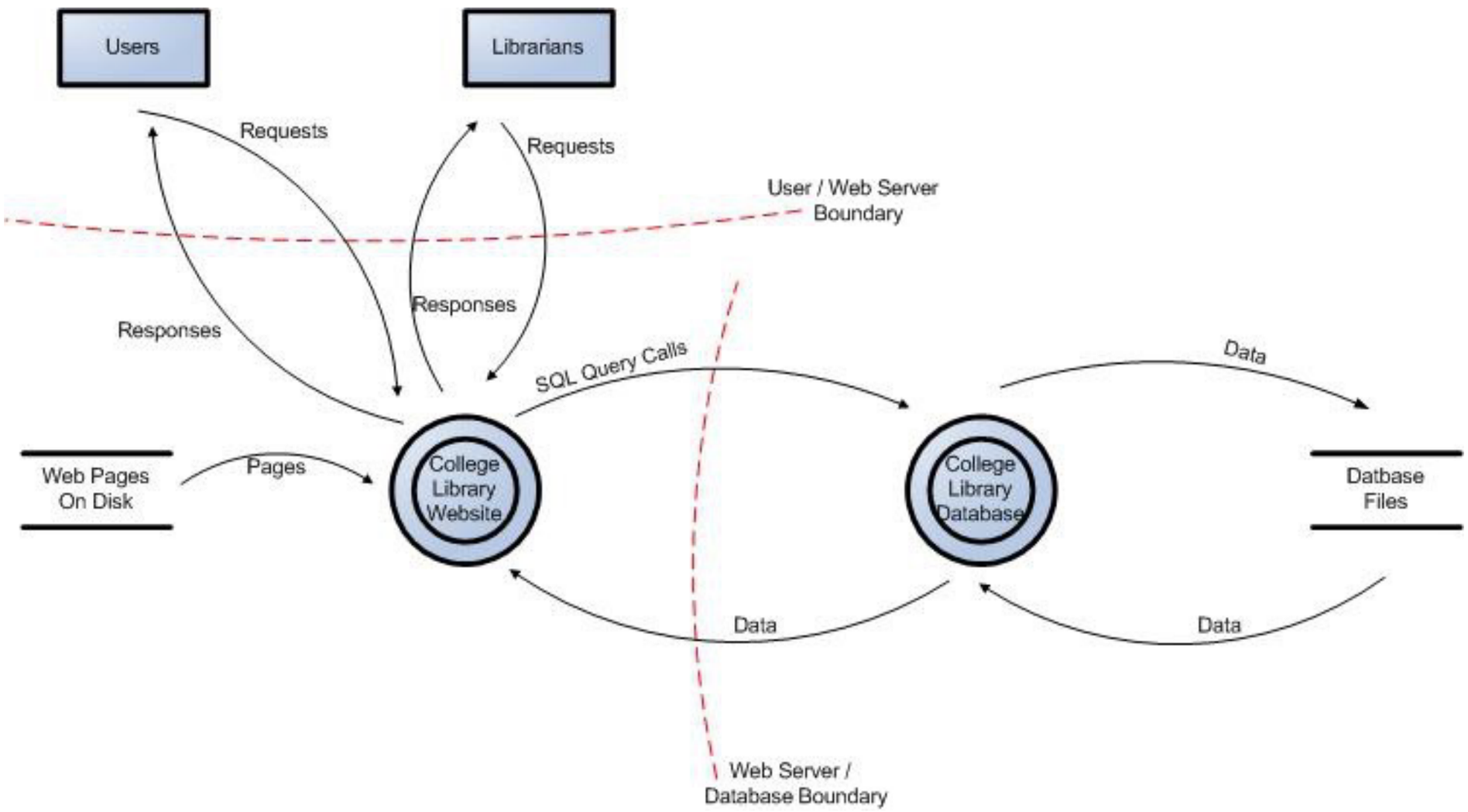
Threat Modeling

- Roughly, we can divide this into three steps:
  - System understanding
  - Threat categorization
  - Countermeasures and mitigation

System Understanding

- To properly protect a system, you need to understand it:
  - Identify assets that need protecting
  - Look at ways the system can be used and assets can be accessed
  - Figure out what rights should be granted to what assets and classes of user
  - Identify *privilege boundaries* – places where a program or user changes their privilege level

Example System Diagram



Source: OWASP

Threat Categorization

- Look at the system from an attacker's point of view
- What goals might an attacker have?
- How could they achieve these goals?
- May help to use a threat categorization such as **STRIDE**: **S**poofing, **T**ampering, **R**epudiation, **I**nformation Disclosure, **D**enial of Service, **E**levation of Privilege

Mitigation

- For each of the threats to some asset, come up with a plan for **mitigating** or nullifying the threat, for example:
  - Attacker might guess someone's password => enforce password complexity requirements
  - Attacker might snoop on network traffic => encrypt data that is sent on the network

Security

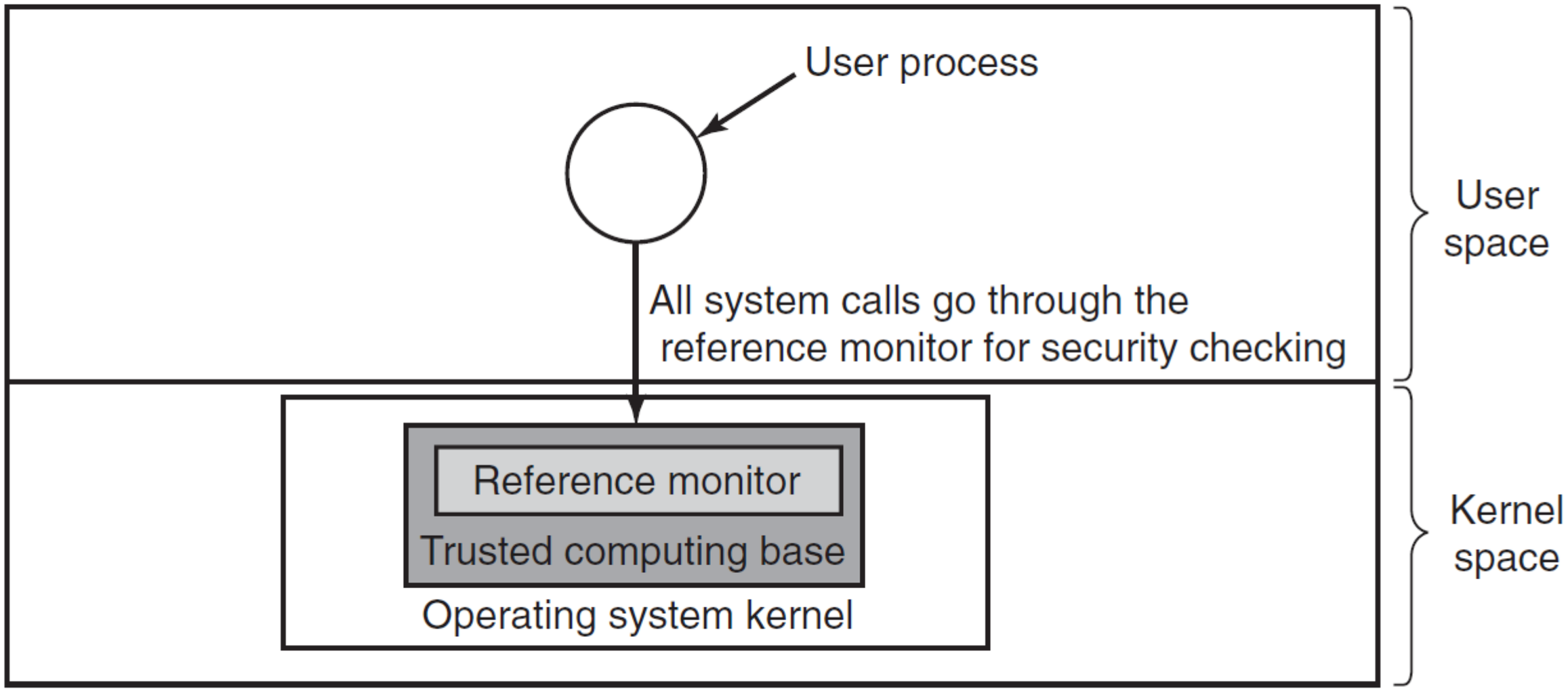
- The Security Environment
- OS Security
- Formal Models of Secure Systems
- Basics of Cryptography
- Authentication
- Software Security

Trusted Computing Base

- One strategy for building secure operating systems is to organize them into *trusted* and *untrusted* components
- The goal is that if the *trusted* component performs according to its specification, then some specific set of guarantees about security must hold
- A *reference monitor* checks all accesses between the trusted and untrusted components

Trusted Computing Base

- The TCB typically consists of:
  - Most of the hardware except (I/O devices that don't affect security)
  - Part of the kernel (process creation, proc. Switching, mem management, file I/0)
- In a secure design the TCS will be separate from the rest of the OS


User process
All system calls go through the
reference monitor for security checking
Reference monitor
Trusted computing base
Operating system kernel
User
space
Kernel
space

Aside: Bugs & Program Size

- One rule of thumb is that as program size increases, the number of bugs increases as well
- It's harder to reason about code the more of it there is
- Therefore, if we want to be confident in our trusted computing base, we should work to *minimize* the amount of code in it
- How big is the TCB(Trusted Computed Base) for widely-used operating systems?

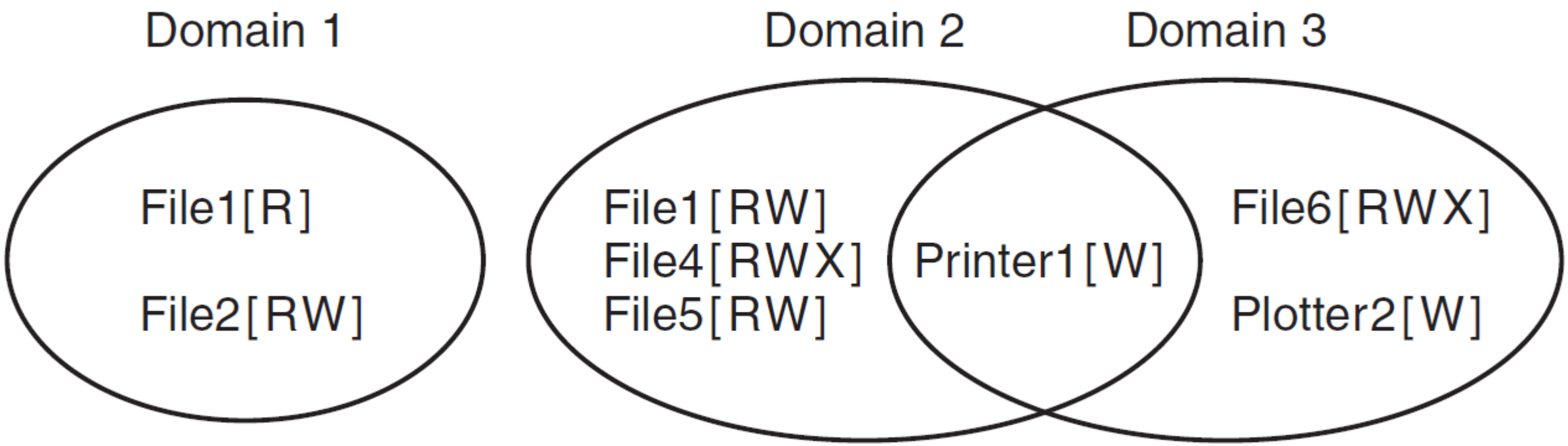
Access Control: Resources

- Systems tend to have resources that need to be protected
  - Hardware: CPUs, memory, disk drives, ...
  - Software: processes, files, databases, ...
- We'll refer to these generally as "objects" for now

Protection Domain

- A **protection domain** is a set of (object, rights) pairs
- A **right**, in this context, means an operation that can be performed on an object
- So, for example, a protection domain might correspond to a user, and the set of objects they have access to
- Or, a *group* of users that all share the same rights

Protection Domain



Principle of Least Privilege

- One principle for designing secure systems is *principle of least privilege*, also called the *principle of least authority*
- Idea: Give the minimal set of rights to an object.
- Seems obvious, but is often violated in practice

Processes and Protection Domains

- At any given time, a process operates in one protection domain
  - I.e., there is some specific set of objects that it has permission to perform some actions on
- Processes can also typically switch between protection domains as they run
  - The rules for when and how they do this vary widely between different operating systems

The UNIX Protection Model

- The protection domain of a process in UNIX is defined by its user id (UID) and group id (GID)
- The objects are files (including special files like hardware devices)

Cryptographic Capabilities

- Cryptography provides us with a way to store a capability wherever we like (even in user space) while protecting it from modification

- Suppose we have a cryptographic one-way function:
  f(Data, Secret) => Output (fixed length)

- Where it is computationally infeasible to figure out Secret if we only have Output

Cryptographic Capabilities

- To give a process a capability, kernel gives out
  ObjectId, Rights, Token
  where Token = f(ObjectId+Rights, Secret)
- Now when the process wants to access something, it includes the capability, including the token
- The kernel can now compute f(ObjectId+Rights, Secret) and compare it to the Token
- If they match, we know the ObjectId and Rights have not been modified

Cryptographic Capabilities

- The nice thing about using cryptography in this way is that we can store the capability *anywhere*
- In particular, capabilities can even be given out to remote machines that we don't control
- This makes them ideal for ***distributed systems***
- Example: the Kerberos protocol uses capabilities this way (though it calls them *tickets*)

Security

- The Security Environment
- OS Security
- Controlling Access to Resources
- Formal Models of Secure Systems
- Basics of Cryptography
- Authentication
- Software Security

Multi-Level Security

- The usual way to do mandatory access control is by splitting users and information into access levels
- Classic example: unclassified, secret, top secret
- However, there still has to be some policy for allowing information to move between levels so work can get done

Bell-LaPadula

- Designed to formally model the US Department of Defense security policy
- Users and data are assigned security levels from low to high
- Then we have the policy:
  - Users at a lower level cannot read information at a higher level (**no read up**)
  - Users at a higher level cannot send information to a lower level (**no write down**)

Multilevel Security Systems

- Levels are confidential, Secret and Top Secret
- Compartments: Crypto, Subs, NoForn (no foreign nationals)
- To read a document, you must have at least as high a clearance level and you must be cleared for each compartment
- Systems that support this level are called Multilevel security systems.

Example

- Pat is cleared for **Secret**, subs
- Chris is cleared for **Top Secret**, planes
- We have the following files:

warplan	**Top Secret**	Troops, Subs, Planes
Runway	**Confidential**	Planes
Sonar	**Top secret**	Subs
Torpedo	**Secret**	Subs

- Who can read which file?

Covert Channels

- Do these models actually guarantee that in practice, unauthorized information can't leak between levels?
- **No!** It turns out there are many more ways to transfer information than the authorized channels
- For example, a user at one security level could send sensitive information to a lower level by performing more or less computation
  - Resulting delays could be observable to lower levels, transmitting information

Steganography

- Even if we allow a human to inspect all data passed between levels, we can still have covert channels
- The way to do this is with *steganography* – hiding data inside other data
- Essentially, make changes that are not observable unless you know what you're looking for, and use those changes to encode information

To the Members of the California State Assembly:

I am returning Assembly Bill 1176 without my signature.

For some time now I have lamented the fact that major issues are overlooked while many unnecessary bills come to me for consideration. Water reform, prison reform, and health care are major issues my Administration has brought to the table, but the Legislature just kicks the can down the alley.

Yet another legislative year has come and gone without the major reforms Californians overwhelmingly deserve. In light of this, and after careful consideration, I believe it is unnecessary to sign this measure at this time.

Sincerely,

Arnold Schwarzenegger

To the Members of the California State Assembly:

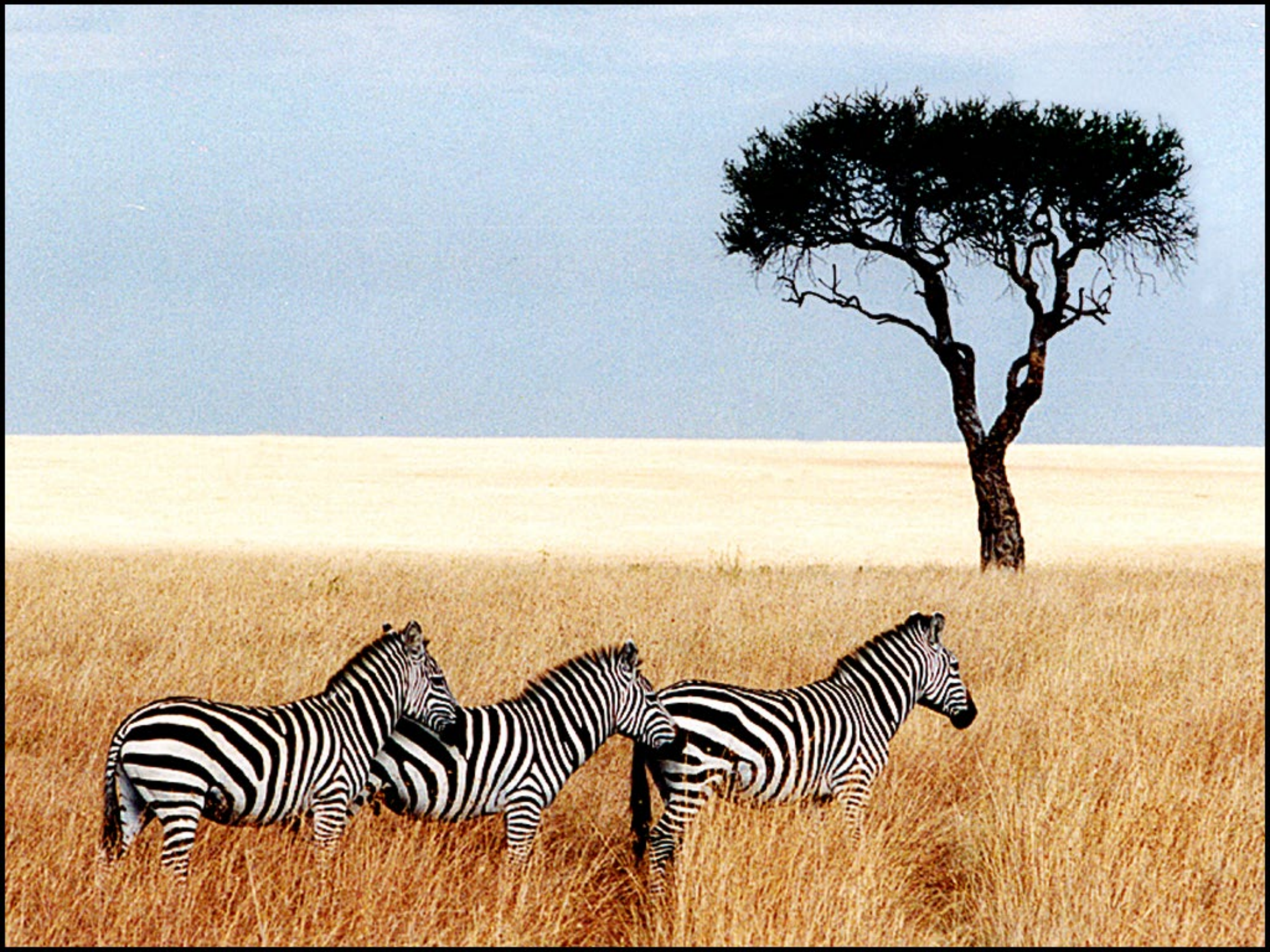
I am returning Assembly Bill 1176 without my signature.

For some time now I have lamented the fact that major issues are overlooked while many unnecessary bills come to me for consideration. Water reform, prison reform, and health care are major issues my Administration has brought to the table, but the Legislature just kicks the can down the alley.

Yet another legislative year has come and gone without the major reforms Californians overwhelmingly deserve. In light of this, and after careful consideration, I believe it is unnecessary to sign this measure at this time.

Sincerely,

Arnold Schwarzenegger



Steganography

- How does this work? Suppose each color is represented by three bytes (red, green, blue)
- Take the least significant bit of each byte and use it to store your own information
- This only adds or subtracts 1 from each color value – not visually detectable
- So we can now get 1 bit of secret information per byte of public information!

Security

- The Security Environment
- OS Security
- Controlling Access to Resources
- Formal Models of Secure Systems
- Basics of Cryptography
- Authentication
- Software Security

Cryptography

- Cryptography has been around a *long* time – at least since the Greeks, and probably earlier
- But it has changed a lot since then
- Particularly since World War II, many advances have been made in both codemaking and codebreaking
- (Right now, the codemakers are winning!)

Kerckhoffs's Principle

- Basic principle: it should not matter if the cryptographic *algorithm* is known by everyone
- The only secret information should be a *secret key* chosen by the participants
  - Less information to keep secret
  - If the key is revealed, we can just change it
- Restated by Claude Shannon: "The enemy knows the system"

Secret Key Cryptography

- Two parties who want to communicate agree on a *secret key* shared between them
- Then they use *encryption* and *decryption* functions:
  - E(Data, Key) = Encrypted Data
  - D(Data, Key) = Decrypted Data
  - D(E(Data, Key), Key) = Data

Example: Caesar Cipher

- Algorithm: take each letter and shift it forward in the alphabet by n letters
- Secret key: the number to shift by (1-25)
- For example, ROT13: shift by 13 letters
  - ATTACK AT DAWN
  - NGGNPX NG QNJA

Secret Key Cryptography

- Modern secret key crypto is much more sophisticated
- Algorithms like AES (the Advanced Encryption Standard) employ substitutions and permutations so that the output has no relationship to input unless the secret is known
  - As far as we know! We have been wrong before...
- Keys are generally 128 or 256 **random** bits – much too large to try all combinations

Public Key Cryptography

- Secret key cryptography was the only kind that existed until the 1970s
- It's inconvenient! Two parties have to somehow securely transmit (or agree on) a secret key
- Particularly on the Internet, this is impractical
  - Imagine having to visit an Amazon office to get a secret key in order to shop online...

Public Key Cryptography

- Instead of a single secret key, we now have a key with two parts: a public key and a private key
  - E(PubKey, Data) = Encrypted Data
  - D(PrivKey, Data) = Decrypted Data or Data
  - **D(PrivKey, E(PubKey, Data)) = Data**
- So now to communicate securely, we just need to know someone's public key

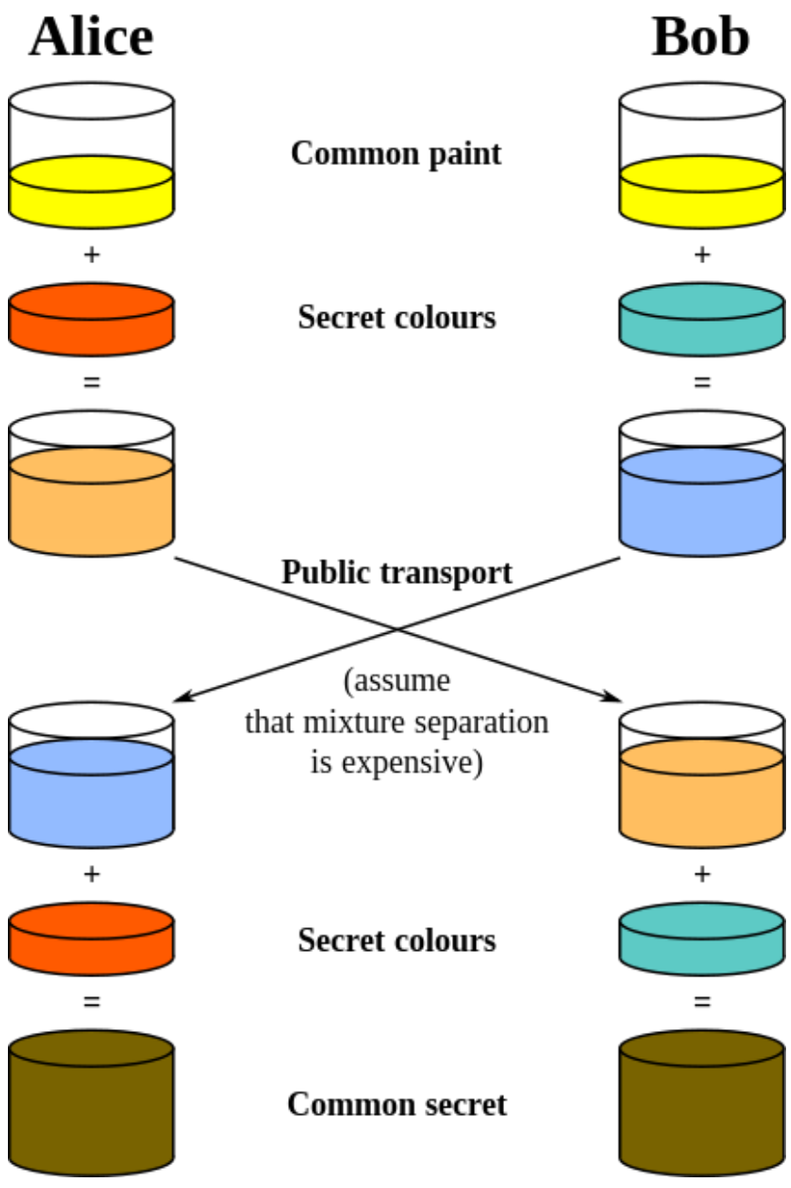
Public Key Crypto Algorithms

- Some examples:
  - RSA (1977)
  - Elliptic Curve Algorithms

Key Exchange

- A slightly different twist on this: can we use only public communications to allow two people to agree on a secret key?
- Surprisingly, the answer is yes!
- Whitfield Diffie and Martin Hellman first showed this was possible in 1976

Diffie-Hellman by Analogy



A.J. Han Vinck, Introduction to public key cryptography, p. 16

Hash Functions

- We saw before that one-way functions can let us do useful things like store capabilities
- Basic idea: Hash(x) = y
  - **Preimage resistance:** It should be very difficult to take y and figure out x
  - **Second preimage resistance:** It should be very difficult to find another value z where Hash(z) = y
  - **Collision resistance:** It should be very difficult to find x1 and x2 such that Hash(x1) = Hash(x2)
- Examples: MD5, SHA-1, SHA-3

Security

- The Security Environment
- OS Security
- Controlling Access to Resources
- Formal Models of Secure Systems
- Basics of Cryptography
- Authentication
- Software Security

Authentication

- To prove you are who you say you are, we usually use one of three things:
  - Something you know
  - Something you have
  - Something you are

Passwords

- "Something you know" – one of the simplest and oldest forms of authentication, and still ubiquitous
- Generally easy to change
- Very easy to implement, since you can just ask for a short string and compare it with the one you have stored

Password Storage

- It's generally a bad idea to store passwords directly
- If someone gets ahold of the password file, they can read all passwords
- This is particularly bad since people tend to re-use their passwords in multiple places...

Password Storage

- Instead, we use a cryptographic hash function on the password
- We compute (for example):
  - SHA1("goodpassword") = 7e5ce399fbe3713ec7f6aae370448cdf990f0aaa
- Then we only store 7e5ce399fbe3713ec7f6aae370448cdf990f0aaa
- Now to check the password someone entered, we hash it and compare the hashes instead

How Passwords are Broken

- Assuming they only have a file with password hashes, how can an attacker find out the original passwords?
- Password hashes nowadays are broken primarily in two ways:
  - **Precomputation**: Spend a lot of time beforehand computing the hash of every possible password, then just look up each hash in your table
  - **Dictionary Attacks**: Guess the most common passwords (e.g. using a dictionary), hash them, and compare

Password Salts

- To avoid precomputation attacks, we can use a *salt*
- Instead of computing and storing Hash(Password), we instead:
  - Generate a random string (a **salt**)
  - Compute Hash(Salt+Password)
  - Store on disk Salt, Hash(Salt+Password)
- Now to do a precomputation, you have to store not just the hash of every possible password (hard but doable) but the hash of every possible password+salt (way too much space)

Time- and Memory-Hard Password Hashing

- Cryptographic hash functions were developed to be *fast*
- This is good for many uses of them, but bad for passwords: it lets an attacker try **billions** of passwords per second
- To avoid this, people have developed hash algorithms that are deliberately very slow (and cannot be sped up)
  - One example is **bcrypt**
- We may also want it to take a lot of memory
  - A newer algorithm called **scrypt** does this

One-Time Passwords

- An alternative to having a single password is to use a different password every time
- Even if an attacker eavesdrops on a connection and learns the password, it doesn't do them any good
- A password list can be exchanged securely (e.g., banks sometimes send them in the mail)

Nordea Bank one-time password list

Card no. 9008759225

01-IU 4455	11-LN 3207	21-GR 2807	31-WD 7558
02-OH 7438	12-UF 6838	22-RX 1323	32-WK 7765
03-NU 2365	13-SL 7027	23-PJ 7191	33-KY 0452
04-II 8859	14-RN 7894	24-WZ 6752	34-MF 0965
05-IQ 0388	15-BE 1806	25-XQ 1597	35-CN 4260
06-WQ 3572	16-ZL 1769	26-IM 1498	36-TZ 5047
07-SJ 7844	17-QM 3891	27-MI 0762	37-SM 7916
08-IV 6424	18-TP 9892	28-TM 0987	38-KQ 6426
09-GK 9623	19-US 1854	29-PD 5288	39-ES 5992
10-WU 5578	20-TH 5502	30-UH 5939	40-VJ 3515

41-GI 7146	51-QH 8027	61-ZT 0271	71-FD 4730
42-FP 3297	52-CJ 6502	62-KG 2048	72-VV 9845
43-VB 9276	53-JC 5925	63-SI 0550	73-GM 7628
44-VY 6679	54-MM 3286	64-UT 4694	74-YZ 1691
45-SW 5187	55-RT 2311	65-UU 2301	75-CG 5697
46-AV 6100	56-BT 1704	66-RW 8490	76-SF 2107
47-GJ 3536	57-JO 0729	67-ZP 3758	77-DT 7470
48-OC 2813	58-YM 6206	68-DR 8424	78-JU 0047
49-ZM 9175	59-GH 3024	69-BL 8702	79-ON 3292
50-NT 9622	60-JQ 3697	70-LP 7668	80-YN 9361

81-LA 6865	91-NR 5144	101-WJ 7731	111-PV 1825
82-DK 1261	92-WY 0917	102-KN 6907	112-QF 4198
83-VS 1837	93-RL 2605	103-IL 8153	113-GU 7578
84-IR 2804	94-GO 0523	104-DE 0478	114-EK 8973
85-ZX 2250	95-RE 3860	105-HF 7664	115-YH 2589
86-MO 9659	96-XE 8676	106-XZ 0754	116-YD 6548
87-PR 5720	97-ZG 4926	107-UC 0587	117-TO 0547
88-LB 0288	98-TJ 9531	108-LK 9936	118-XO 0678
89-GE 5036	99-CP 3886	109-HW 7655	119-FJ 8151
90-CZ 4174	100-GS 9595	110-AJ 9792	120-OL 0586

Lamport Hash Chain

- Instead of mailing out passwords each time, we can get many of the benefits using a hash chain
- The idea is to use a one-way function to create a sequence of hashes
- If you repeatedly apply a hash to some starting value, you get a sequence that is:
  - Easy to compute starting from the seed going forward
  - Hard to compute starting from a hash value and going backward

Lamport Hash Chain

- Initially, user chooses a password p
- Compute $f^k(p) = f(f(f(...f(p))))$ and store this value on the server
- To log in, user computes and sends $h = f^{k-1}(p)$
- Server checks that f(h) matches its stored value, and then replaces the current stored value with h

Challenge-Response

- Another way to avoid sending a password directly over the network is *challenge response* authentication
- The idea is that the server sends a ***challenge*** (a random value)
- Client then uses their secret and the challenge to compute a ***response***
- Server can now verify that the response is the correct one for that challenge

Challenge-Response

- One simple way to implement challenge response is to once again use a one-way function
- Client and server each know some secret *s* (say, a password)
- Server computes a random *r* and sends it to the client
- Client computes f(r+s) and sends it to the server
- Server can do the same computation to verify
- Value sent over the network is useless to an attacker since the challenge (*r*) will not be reused

Authentication

- To prove you are who you say you are, we usually use one of three things:
  - Something you know
  - Something you have
  - Something you are

Physical Tokens

- Oldest examples: physical keys
- Basically – something you physically possess that grants you access to something
- There are lots of these now: ATM cards, smart cards, RFIDs, secure tokens, smartphones

Smart Cards

- Credit card sized
- Contain a chip that can do some simple cryptographic computations and stores a secret key
- The reader can then ask the chip to do cryptographic operations *without* revealing the key to the reader

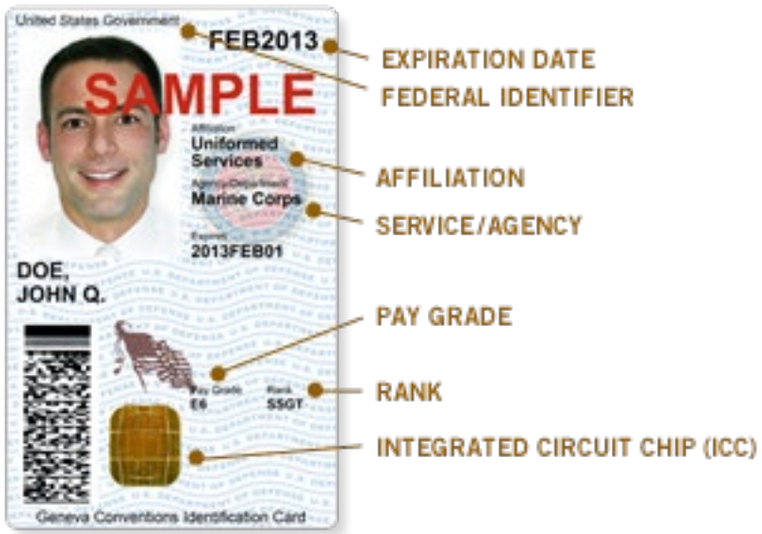
RFID Authentication

- Radio communications can be used instead of direct contact
- When you put the card near a reader, the reader actively probes
- The card can collect energy from the incoming radio waves, compute a response, and send it back

The Great Seal Bug

- An early predecessor to RFID technology was the *great seal bug*
- This was a carved wooden seal presented by the Soviets to the US in 1945
- If radio waves at the right frequency were sent to the device, it activate the circuit and send sound back
- Entirely passive listening device – very hard to detect!

Physical Tokens



Authentication

- To prove you are who you say you are, we usually use one of three things:
  - Something you know
  - Something you have
  - Something you are

Biometrics

- Idea: measure something intrinsic to a person that is different for everyone
- Things you can measure (partial list):
  - Fingerprints
  - Retina pattern
  - Voiceprint
  - Gait
  - Keystroke timing
  - Finger length
  - Face recognition
  - DNA?

Biometrics

- Two phases: *enrollment* and *verification*
- Enrollment consists of taking the measurements of a user and converting them into digital form
  - Often measurements are taken multiple times to account for measurement error and natural variation
- At verification time, redo the measurement, and decide if it's **close enough** (will almost never be exactly the same)
- Note the tradeoff here: risk of rejecting a valid user vs accepting an invalid user

Biometric Downsides

- Many biometric features are not very stable
  - Voiceprint may change if you get a cold
  - Gait may change if you twist your ankle
- Some features may not be very acceptable to users
  - E.g., blood sample collection

Biometric Revocation Problem

- The biggest problem with biometrics is that they are not *revocable*
- If my password is stolen, I get a new password
- If my fingerprints are stolen, ???

Also vulnerable to attacks

IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, VOL. 12, NO. 9, SEPTEMBER 2017 2013

MasterPrint: Exploring the Vulnerability of Partial Fingerprint-Based Authentication Systems

Aditi Roy, *Student Member, IEEE*, Nasir Memon, *Fellow, IEEE*, and Arun Ross, *Senior Member, IEEE*

***Abstract*— This paper investigates the security of partial fingerprint-based authentication systems, especially when multiple fingerprints of a user are enrolled. A number of consumer electronic devices, such as smartphones, are beginning to incorporate fingerprint sensors for user authentication. The sensors embedded in these devices are generally small and the resulting images are, therefore, limited in size. To compensate for the limited size, these devices often acquire multiple partial impressions of a single finger during enrollment to ensure that at least one of them will successfully match with the image obtained from the user during authentication. Furthermore, in some cases, the user is allowed to enroll multiple fingers, and the impressions pertaining to multiple partial fingers are associated with the same identity (i.e., one user). A user is said to be successfully authenticated if the partial fingerprint obtained during authentication matches any one of the stored templates. This paper investigates the possibility of generating a "MasterPrint," a synthetic or real partial fingerprint that serendipitously matches one or more of the stored templates for a significant number of users. Our preliminary results on an optical fingerprint data set and a capacitive fingerprint data set indicate that it is indeed possible to locate or generate partial fingerprints that can be used to impersonate a large number of users. In this regard, we expose a potential vulnerability of partial fingerprint-based authentication systems, especially when multiple impressions are enrolled per finger.**

***Index Terms*— Authentication, biometrics, computer security, dictionary attack, fingerprint recognition, hill climbing, mobile applications, mobile device authentication, partial fingerprint.**

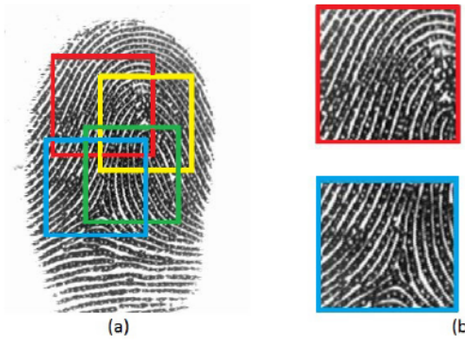


Fig. 1. A set of partial fingerprints (b) extracted from t

Companies such as Apple and Samsung
fingerprint sensors for authenticating users
The European Association for Biometrics con
of the iPhone 5s in September 2013 as hera
shift [3]. This shift is now fueling a secon
in user authentication that uses fingerprints f
other financial transactions. In addition to
services by Apple, Samsung and others, a
a fingerprint scanner has been developed
MasterCard.

Although, from a usability point of view, fi
tication is being touted as a much awaite

Evolutionary Methods for Generating Synthetic MasterPrint Templates: Dictionary Attack in Fingerprint Recognition

Aditi Roy, Nasir Memon, Julian Togelius
New York University Tandon School of Engineering, Brooklyn, NY, USA
ar3824@nyu.edu, memon@nyu.edu, julian@togelius.com

Arun Ross
Michigan State University, East Lansing, MI, USA
rossarun@cse.msu.edu

Abstract

Recent research has demonstrated the possibility of generating "Masterprints" that can be used by an adversary to launch a dictionary attack against a fingerprint recognition system. Masterprints are fingerprint images that fortuitously match with a large number of other fingerprints thereby compromising the security of a fingerprint-based biometric system, especially those equipped with small-sized fingerprint sensors. This work presents new methods for creating a synthetic MasterPrint dictionary that sequentially maximizes the probability of matching a large number of target fingerprints. Three techniques, namely Covariance Matrix Adaptation Evolution Strategy (CMA-ES), Differential Evolution (DE) and Particle Swarm Optimization (PSO), are explored. Experiments carried out using a commercial fingerprint verification software, and public datasets, show that the proposed approaches performed quite well compared to the previously known MasterPrint generation methods.

The vulnerability of such a partial fingerprint-based user authentication system was recently investigated by Roy et al. in [9]. Their work showed that it is possible to launch a dictionary attack with substantial success by using a set of carefully chosen "MasterPrints". Masterprints are partial fingerprint impressions that fortuitously match with a high proportion of other fingerprints corresponding to different subjects. Therefore, an adversary can generate a set of fingerprint spoofs inscribed with Masterprints in order, for example, to unlock the smartphone of a large number of subjects.

In [9], two approaches were proposed to generate MasterPrints, namely "Sampled MasterPrint" (SAMP) and "Synthetic MasterPrint" (SYMP). In the first approach, the Masterprints - known as SAMPs - were sampled from a fixed fingerprint dataset. In this case, each fingerprint in the dataset was matched with all other fingerprints in the dataset, and the ones with the highest Impostor Match Rates were selected as Masterprints. In the second approach, Masterprints were created synthetically by apply-

Matthew Green
@matthew_d_green

Following

I woke this morning to find my 7 y/o levering my finger onto the TouchID sensor of my phone. Maybe time to go back to passwords.

RETWEETS **2,466** LIKES **1,834**

8:30 AM - 25 Nov 2014

Security

- The Security Environment
- OS Security
- Controlling Access to Resources
- Formal Models of Secure Systems
- Basics of Cryptography
- Authentication
- Software Security

Software Security

- One final aspect to OS security is *software security* – identifying and preventing programming flaws that could let an attacker take control

- Many of these are caused by the fact that languages currently in use are not *memory safe* – it is possible to write to data outside of program variables

- A big offender here is C/C++

Stack Buffer Overflows

- We saw one of these earlier in the semester
- Recall the standard stack frame:
  - Local variables
    Saved frame pointer (optional)
    Return address
- If we try to store too much data in a stack variable (e.g. a character array) we will overwrite the frame pointer and return address
- When the ret instruction is executed, it will jump to somewhere controlled by user input

Classic Buffer Overflow

```
#include <stdio.h>
#include <string.h>

int main(int argc, char* argv[]) {
    char buf[64];
    strcpy(buf,argv[1]);
    printf("Input:%s\n",buf);
    return 0;
}
```

Demo

- Video:
- Running a Buffer Overflow Attack - Computerphile - YouTube
- Script:
- Buffer overflow demonstration in Kali Linux, based on the Computerphile video (github.com)

Operating System Defenses

- OSes can be designed to make problems in user-level applications harder to exploit
- These generally don't make attacks on software impossible, but they can make them much more difficult
- This is something of an arms race – defenders come up with new mechanisms, attackers find ways around them

Stack Canaries / Cookies

- Idea: put a special value in between local variables and the return address so that overflowing a local buffer can be detected
- Upon entering the function, set a randomly-generated *cookie* value on the stack and store a backup copy elsewhere
- Before executing a ret, check the stack cookie value against the backup and raise an error if it fails

Address Space Layout Randomization

- Exploiting a buffer overflow typically requires knowing about the precise layout of memory
- For example, we may need to know where the stack is located, or where a certain library has been loaded
- Thus, to make attackers' lives more difficult, we can place the program, stack, and libraries at random locations each time the program starts

ASLR and Address Space

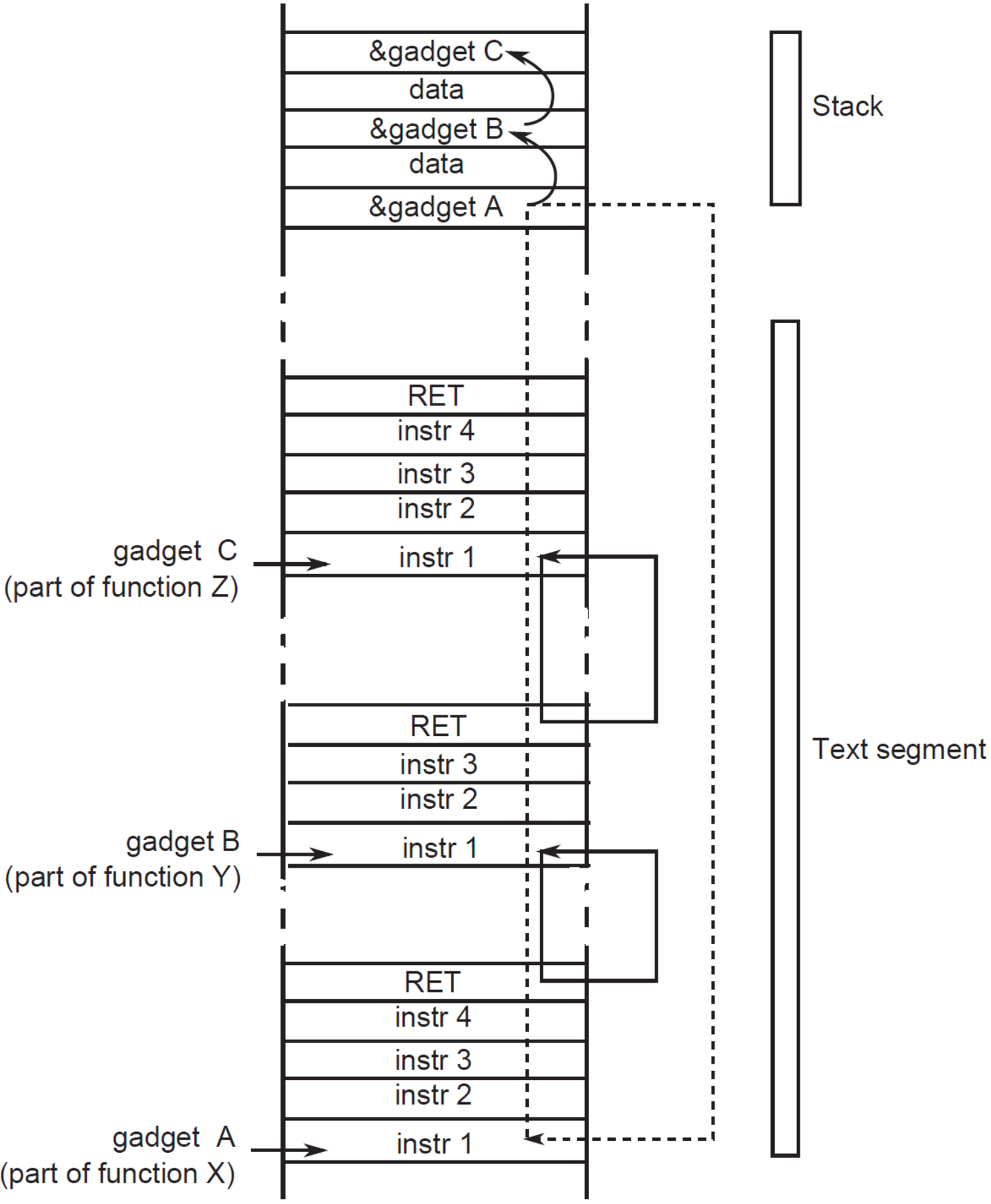
- We can estimate the amount of randomness provided by ASLR by counting the number of possible locations to load things
- On a 32-bit system, address space is not very large, so there are not very many ways to randomize
- In 2004, researchers showed that on 32-bit systems, ASLR only has about 16 bits of entropy (65,536 possible values)
- The correct location can be guessed in a matter of seconds by just trying each possibility

DEP / NX / W⊕X

- Another defense is to try and make sure that even if an attacker can overflow a buffer and change the return address, they can't execute code
- In the previous example, attacker code was placed into a stack buffer
- So, simple solution: don't allow data to be executable!
- Generally this requires hardware support
  - NX bit in x86 page protections

Return-Oriented Programming

- Despite DEP, attackers can still run code!
- Instead of trying to execute their *own* code, attackers can change the return address to point to existing code in memory
- By setting up values on the stack, they can bounce around executing tiny snippets of code ending in ret
- Thus, by chaining these together, *arbitrary* computation can be performed – without executing anything marked as data



(a)

Example gadgets:

Gadget A:
- pop operand off the stack into register 1
- if the value is negative, jump to error handler
- otherwise return

Gadget B:
- pop operand off the stack into register 2
- return

Gadget C:
- multiply register 1 by 4
- push register 1
- add register 2 to the value on the top of the stack and store the result in register 2

(b)

Dangling Pointers

- An increasingly common and exploited class of vulnerability is the *dangling pointer* or *use after free* vulnerability
- Scenario: programmer frees an object, but then continues to use the pointer afterward
- Problem: new allocations may use the same space, placing a *different* object at the same location
- If an attacker can manage to get his data placed into that freed space, can manipulate the program into executing his code

Use After Free

```
int *A = (int *) malloc (128);                /* allocate space for 128 integers */
int year_of_birth = read_user_input ( ); /* read an integer from standard input */
if (input < 1900) {
    printf ("Error, year of birth should be greater than 1900 \n");
    free (A);
} else {
    ...
    /* do  something interesting with array A */
    ...
}
... /* many more statements, containing malloc and free */
A[0] = year_of_birth;
```

TOCTOU

- One final class of attacks is a *time of check to time of use* attack
- These arise any time you have a situation where
  1. A security check on some object is made
  2. The program does something with that object
  3. An attacker can intervene in between 1 & 2 and cause it to operate on a different (attacker-controlled) object

TOCTOU Example

```
int fd;
if (access ("./my_document", W_OK) != 0) {
    exit (1);
fd = open ("./my_document", O_WRONLY)
write (fd, user_input, sizeof (user_input));
```

TOCTOU Attack

- To exploit this program, the attacker runs the program with a my_document he controls
- After the access check is done, he deletes my_document and creates a symbolic link from my_document to some sensitive file
  - Getting the timing right here may be tricky!
- The program now opens and writes to my_document, but that now results in writing to the sensitive file

Preventing TOCTOU

- In general, the only way to prevent TOCTOU attacks is through careful API design
- In this case, we want to make sure that the file cannot be changed between the access check and the call to open
- One way to do this is to open the file and then perform access checks on the file descriptor (which can't be changed by the attacker) rather than the file (which can)

Spectre Bug(2018)

- Spectre attack explained like you're five - YouTube

Meltdown

- Meltdown explained like you're five - YouTube